



THE LEARNING HUB
COMPUTER CENTRE



BCA
IV SEMESTER
BACHELOR OF
COMPUTER
APPLICATIONS



Let's Go

PYTHON PROGRAMMING

THE SMART CHOICE FOR BCA STUDENTS

“ LEARN TODAY,
CODE TOMORROW, LEAD FOREVER ”

WHY PYTHON?

- ✓ Easy to Learn
- ✓ Readable Syntax
- ✓ High-Level Language
- ✓ Cross-Platform
- ✓ Large Community Support
- ✓ Versatile & Powerful



FEATURES

- Simple & Easy Syntax
- Interpreted Language
- High Performance
- Huge Standard Library
- Open Source & Free

BCA RELEVANCE

- Helps in Software Development
- Useful in Data Science & AI
- Supports Web Development
- Strong in Database Connectivity
- Ideal for Mini & Major Projects
- Enhances Problem Solving Skills

APPLICATION AREAS

- Web Development
- Data Science & Analytics
- Machine Learning & AI
- Automation & Scripting
- Game Development
- Cyber Security

`</>` PYTHON BASIC SYNTAX

```
1 # Hello World in Python
2 print("Hello, World!")
3
4 name = "BCA Student"
5 print("Welcome to Python
6       Programming!")
```



BUILD LOGIC.
CREATE SOLUTIONS.
DRIVE INNOVATION.

CAREER OPPORTUNITIES

- Python Developer
- Data Analyst
- Machine Learning Engineer
- Web Developer
- Automation Tester
- Software Engineer



EXPERT
FACULTY



PRACTICAL
LEARNING



PROJECT
BASED



CERTIFICATE
COURSE

INSPIRE • LEARN • LEAD

INTRODUCTION TO PYTHON

TOPIC 5.1 INTRODUCTION TO PYTHON

Computer को किसी विशेष कार्य को करने के लिए दिए गए निर्देशों के क्रमबद्ध समूह को **Program** कहते हैं, और Computer को इन निर्देशों को निर्दिष्ट करने के लिए उपयोग की जाने वाली भाषा को **Programming Language** कहते हैं। Python एक **High-Level Language** है जिसका उपयोग **Computer Programs** लिखने के लिए किया जाता है। Python अपने निर्देशों को **Machine Language** में बदलने के लिए एक **Interpreter** का उपयोग करती है। **Interpreter Program** के **statements** को एक-एक करके **Process** करता है — पहले **Translate** करता है, फिर **Execute** करता है। यह प्रक्रिया तब तक जारी रहती है जब तक कोई **Error** न आए या पूरा **Program** सफलतापूर्वक **Execute** न हो जाए।

FEATURES OF PYTHON

1. Python एक **High-Level Language** है। यह एक **Free** और **Open-Source Language** है। 2. यह एक **Interpreted Language** है, क्योंकि Python Programs को **Interpreter** द्वारा **Execute** किया जाता है। 3. Python Programs को समझना आसान है क्योंकि इनका **Syntax** स्पष्ट और **Structure** सरल होता है। 4. Python **Case-Sensitive** है। उदाहरण के लिए, **NUMBER** और **number** Python में एक समान नहीं हैं। 5. Python **Portable** और **Platform Independent** है, अर्थात् यह विभिन्न **Operating Systems** और **Hardware Platforms** पर चल सकती है। 6. Python में **Predefined Functions** की एक समृद्ध **Library** है। 7. Python **Web Development** में भी सहायक है। कई लोकप्रिय **Web Services** और **Applications** Python का उपयोग करके बनाई गई हैं। 8. Python **Blocks** और **Nested Blocks** के लिए **Indentation** का उपयोग करती है।

EXECUTION MODE OF PYTHON

Python Interpreter का उपयोग करने के दो तरीके हैं: a) **Interactive Mode** b) **Script Mode**

(A) INTERACTIVE MODE

To work in the interactive mode, we can simply type a Python statement on the `>>>` prompt directly as soon as we press enter, the interpreter executes the statement and displays the result.

(B) SCRIPT MODE

In the script mode, we can write a Python program in a file, save it and then use the interpreter to execute it. Python scripts are saved as files where file name has extension `“.py”`.

TOPIC 5.2 PYTHON KEYWORDS

Keywords आरक्षित (Reserved) शब्द होते हैं। प्रत्येक Keyword का Python Interpreter के लिए एक विशेष अर्थ होता है। हम अपने Program में किसी Keyword का उपयोग केवल उसी उद्देश्य के लिए कर सकते हैं जिसके लिए वह परिभाषित किया गया है। Keywords किसी Program के मूल निर्माण-खंड (Basic Building Blocks) होते हैं। Keywords का उपयोग करके Programmer एक Program विकसित करता है।

True	def	from	nonlocal	while
and	del	global	not	with
as	elif	if	or	yield
assert	else	import	pass	
break	except	in	raise	

TOPIC 5.3 IDENTIFIERS

Identifiers वे नाम होते हैं जो Program में किसी Variable, Function या अन्य Entity को पहचानने के लिए उपयोग किए जाते हैं। Python में Identifier के नामकरण के नियम इस प्रकार हैं: • नाम की शुरुआत Uppercase या Lowercase Alphabet अथवा Underscore (_) से होनी चाहिए। Identifier की शुरुआत किसी Digit से नहीं हो सकती। • इसकी लंबाई कितनी भी हो सकती है। (हालाँकि, इसे छोटा और अर्थपूर्ण रखना उचित है।) • यह कोई Keyword या Reserved Word नहीं होना चाहिए। • Identifiers में !, @, #, \$, % आदि Special Symbols का उपयोग नहीं किया जा सकता।

Example: `avg = (marks1 + marks2 + marks3)/3`

TOPIC 5.4 VARIABLES

Program में किसी Variable को उसके नाम (Identifier) द्वारा विशिष्ट रूप से पहचाना जाता है। Python में Variable किसी Object को संदर्भित करता है — एक ऐसा Item या Element जो Memory में संग्रहीत होता है। Variable का मान एक String (जैसे 'b', 'Global Citizen'), Numeric (जैसे 345) या Alphanumeric Characters का कोई भी संयोजन (CD67) हो सकता है।

COMMENTS

Comments का उपयोग Source Code में कोई टिप्पणी या नोट जोड़ने के लिए किया जाता है। Comments को Interpreter द्वारा Execute नहीं किया जाता। Python में Comment # (Hash Sign) से शुरू होता है। उस # के बाद उसी पंक्ति का सब कुछ Comment माना जाता है और Interpreter इसे Execute करते समय अनदेखा कर देता है।

TOPIC 5.4 DATA TYPES

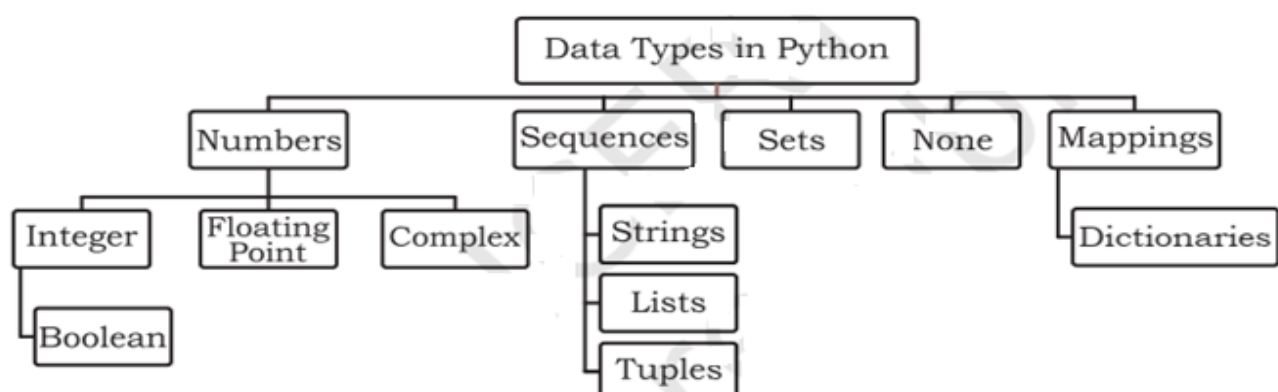
Data Type यह पहचानता है कि Variable किस प्रकार के Data Values को रख सकता है और उस Data पर कौन से Operations किए जा सकते हैं। किसी Programming Language में Variable जिस प्रकार का Data रख सकता है उसे Data Types कहते हैं।

#SIMPLE PYTHON PROGRAM

A=10

B=20

C=A+B print('Sum=',C)



NUMBER DATA TYPE

Type/ Class	Description	Examples
int	integer numbers	-12, -3, 0, 125, 2
float	real or floating point numbers	-2.04, 4.0, 14.23
complex	complex numbers	$3 + 4i$, $2 - 2i$

SEQUENCE DATA TYPE

Python Sequence Items का एक क्रमबद्ध (Ordered) संग्रह है, जहाँ प्रत्येक Item को एक Integer द्वारा Index किया जाता है। Python में उपलब्ध तीन प्रकार के Sequence Data Types हैं: Strings, Lists और Tuples।

(A) STRING

String is a group of characters. These characters may be alphabets, digits or special characters including spaces.

Example >>> str1 = 'Hello Friend' >>> str2 = "452"

(B) LIST

List is a sequence of items separated by commas and the items are enclosed in square brackets []

Example

#To create a list

```
>>> list1 = [5, 3.4, "New Delhi", "20C", 45]
```

```
>>> print(list1)
```

```
[5, 3.4, 'New Delhi', '20C', 45]
```

(C) TUPLE

Tuple is a sequence of items separated by commas and items are enclosed in parenthesis ().

Example

```
>>> tuple1 = (10, 20, "Apple", 3.4, 'a')
```

```
>>> print(tuple1)
```

```
(10, 20, "Apple", 3.4, 'a')
```

SET DATA TYPE

Set Items का एक अव्यवस्थित (Unordered) संग्रह है जिसमें Items अल्पविराम द्वारा अलग किए जाते हैं और Curly Brackets { } में बंद होते हैं। Set, List के समान है, लेकिन इसमें Duplicate Entries नहीं हो सकतीं। एक बार बनने के बाद Set के Elements को बदला नहीं जा सकता।

Example

```
>>> set1 = {10,20,3.14,"New Delhi"}
```

```
>>> print(set1)
```

```
{10, 20, 3.14, "New Delhi"}
```

NONE DATA TYPE

None एक विशेष Data Type है जिसका केवल एक ही मान होता है। इसका उपयोग किसी स्थिति में Value की अनुपस्थिति को दर्शाने के लिए किया जाता है।

Example

```
>>> a=None
```

MAPPING DATA TYPE

Mapping Python में एक Unordered Data Type है। वर्तमान में Python में केवल एक ही मानक Mapping Data Type है जिसे Dictionary कहते हैं।

(A) DICTIONARY

It holds data items in key-value pairs. Items in a dictionary are enclosed in curly brackets { }. Dictionaries permit faster access to data. Every key is separated from its value using a colon (:) sign. The key : value pairs of a dictionary can be accessed using the key. The keys are usually strings and their values can be any data type.

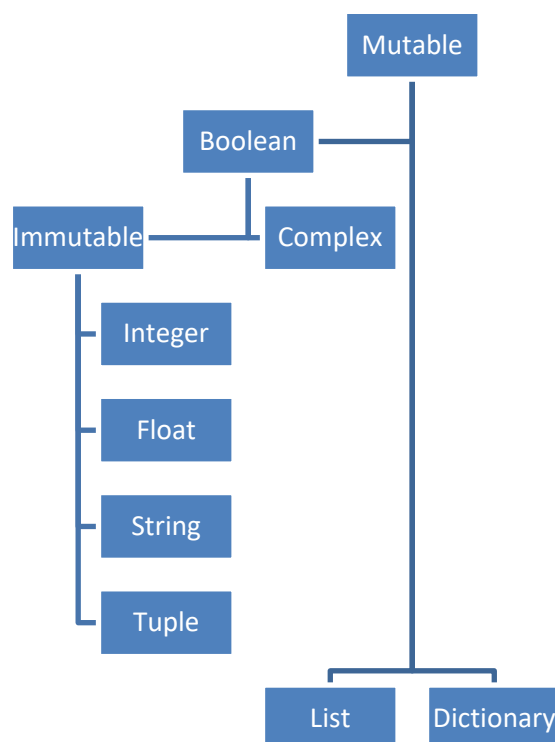
Example

```
>>> dict1 = {'Fruit':'Apple',  
'Climate':'Cold', 'Price(kg)':120}  
>>> print(dict1)  
{'Fruit': 'Apple', 'Climate': 'Cold',  
'Price(kg)': 120}  
>>> print(dict1['Price(kg)'])  
120
```

MUTTABLE & IMMUTABLE DATA TYPES

जिन Variables के मान उन्हें बनाने और Assign करने के बाद बदले जा सकते हैं, उन्हें Mutable कहते हैं।

जिन Variables के मान उन्हें बनाने और Assign करने के बाद नहीं बदले जा सकते, उन्हें Immutable कहते हैं।



TOPIC 5.5 OPERATORS

Operator का उपयोग Values पर विशिष्ट Mathematical या Logical Operation करने के लिए किया जाता है। जिन Values पर Operators काम करते हैं उन्हें Operands कहते हैं। विभिन्न प्रकार के Operators निम्नलिखित हैं:

ARITHMETIC OPERATORS

OPERATOR	OPERATION	DESCRIPTION	EXAMPLE
----------	-----------	-------------	---------

+	Addition	Operator के दोनों ओर के दो Numeric Values को जोड़ता है। इस Operator का उपयोग Operator के दोनों ओर की दो Strings को Concatenate (जोड़ने) के लिए भी किया जा सकता है।	>>> num1 = 5 >>> num2 = 6 >>> num1 + num2 11 >>> str1 = "Hello" >>> str2 = "India" >>> str1 + str2 'HelloIndia'
-	Subtraction	Right Side के Operand को Left Side के Operand से घटाता है।	>>> num1 = 5 >>> num2 = 6 >>> num1 - num2 -1
*	Multiplication	Operator के दोनों ओर के दो Values को गुणा करता है। यदि पहला Operand String है और दूसरा Integer है, तो Left Side के Item को दोहराता है।	>>> num1 = 5 >>> num2 = 6 >>> num1 * num2 30 >>> str1 = 'India' >>> str1 * 2 'IndiaIndia'
/	Division	Left Side के Operand को Right Side के Operand से भाग देता है और Quotient (भागफल) लौटाता है।	>>> num1 = 8 >>> num2 = 4 >>> num2 / num1 0.5
%	Modulus	Left Side के Operand को Right Side के Operand से भाग देता है और Remainder (शेषफल) लौटाता है।	>>> num1 = 13 >>> num2 = 5 >>> num1 % num2 3
//	Floor Division	Left Side के Operand को Right Side के Operand से भाग देता है और Decimal Part हटाकर Quotient लौटाता है। इसे कभी-कभी Integer Division भी कहते हैं।	>>> num1 = 13 >>> num2 = 4 >>> num1 // num2 3 >>> num2 // num1 0
**	Exponent	Operands पर घातांक (Exponential / Power) की गणना करता है।	>>> num1 = 3 >>> num2 = 4 >>> num1 ** num2 81

RELATIONAL OPERATORS

OPERATOR	OPERATION	DESCRIPTION	EXAMPLE
==	Equals to	यदि दोनों Operands के मान बराबर हैं, तो Condition True होती है, अन्यथा False होती है।	>>> num1 == num2 False >> str1 == str2 False
!=	Not equal to	यदि दोनों Operands के मान बराबर नहीं हैं, तो Condition True होती है, अन्यथा False होती है।	>>> num1 != num2 True >>> str1 != str2 True

			>>> num1 != num3 False
>	Greater than	यदि Left Side Operand का मान Right Side Operand के मान से बड़ा है, तो Condition True होती है, अन्यथा False होती है।	>>> num1 > num2 True >>> str1 > str2 True
< >=	Less than (से कम) Greater than or equal to (से बड़ा या बराबर)	यदि Left Side Operand का मान Right Side Operand के मान से कम है, तो Condition True होती है। यदि Left Side Operand का मान Right Side Operand के मान से बड़ा या बराबर है, तो Condition True होती है।	>>> num1 < num3 False >>> str2 < str1 True >>> num1 >= num2 True >>> num2 >= num3 False >>> str1 >= str2 True
<=	Less than or equal to (से कम या बराबर)	यदि Left Operand का मान Right Operand के मान से कम या बराबर है, तो Condition True होती है, अन्यथा False होती है।	>>> num1 <= num2 False >>> num2 <= num3 True >>> str1 <= str2 False

LOGICAL OPERATORS

OPERATOR	OPERATION	DESCRIPTION	EXAMPLE
and	Logical AND	यदि दोनों Operands True हैं, तो Condition True हो जाती है।	>>> True and True True >>> num1 = 10 >>> num2 = -20 >>> bool(num1 and num2) True >>> True and False False >>> num3 = 0 >>> bool(num1 and num3) False >>> False and False False
or	Logical OR	यदि दोनों में से कोई भी एक Operand True है, तो Condition True हो जाती है।	>>> True or True True >>> True or False True >>> bool(num1 or num3) True >>> False or False False
not	Logical NOT	अपने Operand की Logical State को उल्टा करने के लिए उपयोग किया जाता है।	>>> num1 = 10 >>> bool(num1) True >>> not num1 >>> bool(num1)

			False
--	--	--	-------

ASSIGNMENT OPERATORS

OPERATOR	DESCRIPTION	EXAMPLE
=	Right Side Operand से Left Side Operand को Value Assign करता है।	<pre>>>> num1 = 2 >>> num2 = num1 >>> num2 2 >>> country = 'India' >>> country 'India'</pre>

+=	Right Side Operand का मान Left Side Operand में जोड़ता है और परिणाम को Left Side Operand में Store करता है।	<pre>>>> num1 = 10 >>> num2 = 2 >>> num1 += num2 >>> num1 12</pre>
-=	Right Side Operand का मान Left Side Operand से घटाता है और परिणाम को Left Side Operand में Store करता है।	<pre>>>> num1 = 10 >>> num2 = 2 >>> num1 -= num2 >>> num1 8</pre>
*=	Right Side Operand का मान Left Side Operand से गुणा करता है और परिणाम को Left Side Operand में Store करता है।	<pre>>>> num1 = 2 >>> num2 = 3 >>> num1 *= 3 >>> num1 6</pre>
/=	Left Side Operand को Right Side Operand से भाग देता है और परिणाम को Left Side Operand में Store करता है।	<pre>>>> num1 = 6 >>> num2 = 3 >>> num1 /= num2 >>> num1 2.0</pre>
%=	दो Operands पर Modulus Operation करता है और परिणाम को Left Side Operand में Store करता है।	<pre>>>> num1 = 7 >>> num2 = 3 >>> num1 %= num2 >>> num1 1</pre>

//=	दो Operands पर Floor Division करता है और परिणाम को Left Side Operand में Store करता है।	<pre>>>> num1 = 7 >>> num2 = 3 >>> num1 //= num2 >>> num1 2</pre>
**=	Operators पर Exponential (Power) की गणना करता है और मान को Left Side Operand में Store करता है।	<pre>>>> num1 = 2 >>> num2 = 3 >>> num1 **= num2 >>> num1 8</pre>

IDENTITY OPERATORS

OPERATOR	DESCRIPTION	EXAMPLE
is	True का मूल्यांकन करता है यदि Operator के दोनों ओर के Variables एक ही Memory Location की ओर इंगित करते हैं, अन्यथा False।	<pre>>>> num1 = 5 >>> type(num1) is int True >>> num2 = num1 >>> id(num1) 1433920576 >>> id(num2) 1433920576 >>> num1 is num2 True</pre>
is not	False का मूल्यांकन करता है यदि Operator के दोनों ओर के Variables एक ही Memory Location की ओर इंगित करते हैं, अन्यथा True।	<pre>>>> num1 is not num2 False</pre>

MEMBERSHIP OPERATORS

OPERATOR	DESCRIPTION	EXAMPLE
in	यदि Variable/Value निर्दिष्ट Sequence में मिलती है तो True, अन्यथा False लौटाता है।	<pre>>>> a = [1,2,3] >>> 2 in a True >>> '1' in a False</pre>
not in	यदि Variable/Value निर्दिष्ट Sequence में नहीं मिलती तो True, अन्यथा False लौटाता है।	<pre>>>> a = [1,2,3] >>> 10 not in a True >>> 1 not in a False</pre>

PRECEDENCE OF OPERATORS Expression के मूल्यांकन की प्रक्रिया Operators की Precedence पर आधारित होती है। जब किसी Expression में विभिन्न प्रकार के Operators होते हैं, तो Precedence यह तय करती है कि किस Operator को पहले Apply करना है। Higher Precedence वाले Operator को Lower Precedence वाले Operator से पहले Evaluate किया जाता है।

Expression को **Constants, Variables और Operators** के संयोजन के रूप में परिभाषित किया जाता है। **Expression** हमेशा एक **Value** का मूल्यांकन करता है।

Expressions के कुछ उदाहरण नीचे दिए गए हैं।

$Z = X + 2XY + Y$	$Z = (A - B) * (C + D)$
-------------------	-------------------------

क्रम की प्राथमिकता (ORDER OF PRECEDENCE)	OPERATORS	DESCRIPTION
1	**	घातांक (Exponentiation) - शक्ति तक उठाया गया
2	~, +, -	Complement, Unary Plus और Unary Minus
3	*, /, %, //	Multiply, Divide, Modulo और Floor Division
4	+, -	Addition और Subtraction
5	<=, <, >, >=	Relational operators
6	==, !=	Equality Operators
7	=, %=, /=, //=, -=, +=, *=, **=	Assignment operators
8	is, is not	Identity operators
9	in, not in	Membership operators
10	not, or, and	Logical operators

TOPIC 5.6 INPUT & OUTPUT STATEMENTS

`input()` Function का उपयोग User से Input लेने के लिए किया जाता है। `input()` Function User को Data Enter करने के लिए Prompt करता है। यह सभी User Input को String के रूप में Accept करता है। User चाहे Number Enter करे या String, `input()` Function उसे String के रूप में ही मानता है। `input()` का Syntax: `input([Prompt])` है। किसी भी जानकारी का Output `print` Function के माध्यम से होता है। `print` Function एक Message प्रदर्शित कर सकता है और Variable के मान भी Print कर सकता है।

```
# Sample Input Output Program
a=int(input('Enter a Non Decimal Number\t'))
b=float(input('Enter a Decimal Number\t'))
c=input('Enter A Spelling\t')
print('Non Decimal Number:',a)
print('Decimal Number:',b)
print('Word:',c)
```

OUTPUT

Enter a Non Decimal Number456

Enter a Decimal Number 567.78
Enter A Spelling DATA KNOWLEDGE
Non Decimal Number: 456
Decimal Number: 567.78
Word: DATA KNOWLEDGE

TOPIC 5.7 TYPE CONVERSION

एक Data Type Specification को दूसरे में बदलने की प्रक्रिया को Type Conversion कहते हैं। Python Language में दो प्रकार के Type Conversions होते हैं।

IMPLICIT TYPE CONVERSION

इस प्रकार के Conversion को Coercion भी कहते हैं। यह तब होता है जब Data Type Conversion स्वतः (Automatically) Programming Language द्वारा की जाती है, न कि Programmer द्वारा निर्देशित।

```
# Implicit Type Conversion
a=int(input('Enter A Non Decimal Number\t'))
b=float(input('Enter A Decimal Number\t'))
c=a+b
Implicit Type Conversion
```

OUTPUT

Enter A Non Decimal Number 56
Enter A Decimal Number 67.84
Sum: 123.84

EXPLICIT TYPE CONVERSION

Explicit Conversion, जिसे Type Casting भी कहते हैं, तब होती है जब Data Type Conversion Programmer द्वारा जबरदस्ती की जाती है। Explicit Data Type Conversion का सामान्य रूप है: (new_data_type)(expression)। Explicit Type Conversion में जानकारी खोने का खतरा होता है क्योंकि हम किसी Expression को एक विशेष Type का बना रहे होते हैं।

```
# Explicit Type Conversion
a=int(input('Enter A Non Decimal Number\t'))
b=float(input('Enter A Decimal Number\t'))
c=a+int(b)
print('Sum:',c)
```

OUTPUT

Enter A Non Decimal Number 56
Enter A Decimal Number 67.84
Sum: 123

FLOW OF CONTROL

WHAT IS FLOW OF CONTROL

Program में Statements के Execution का क्रम Flow of Control कहलाता है। Flow of Control को Control Structures का उपयोग करके लागू किया जा सकता है। Python दो प्रकार के Control Structures का समर्थन करती है — Selection और Repetition।

TOPIC 6.1 SELECTION

Programming में Decision Making या Selection की अवधारणा को IF_ELSE Statement की सहायता से लागू किया जाता है। विभिन्न Selection Statements इस प्रकार हैं:

- (1) **SIMPLE IF STATEMENT**
- (2) **IF_ELSE STATEMENT**
- (3) **IF_ELIF_ELSE STATEMENT**

SIMPLE IF STATEMENT

In this type of statement only one condition is framed on the repetitive statement. As per one condition a block of statement is executed whether the condition is executed as TRUE. If the condition is true, then the indented statement(s) are executed. The indentation implies that its execution is dependent on the condition. There is no limit on the number of statements that can appear as a block under the if statement.

if Statement का Syntax इस प्रकार है:

```
if condition:
    statement(s)
```

Simple If Statement Representation

```
age = int(input("Enter your age "))
if age >= 18:
    print("Eligible to vote")
```

IF_ELSE STATEMENT

इस प्रकार के Statement में Repetitive Statements पर एक Condition होती है। इस Statement में True और False Block Statements होते हैं। यदि दिया गया Statement True Evaluate होता है तो True Block Statements Execute होते हैं। यदि Condition False है तो False Block Statements Execute होते हैं।

IF..ELSE Statement का Syntax इस प्रकार है:

```
if condition:
    statement(s)
else:
    statement(s)
```

IF_ELIF_ELSE STATEMENT

इस प्रकार के Statement में एक साथ कई Conditions Check की जाती हैं। इसमें अनेक विकल्प होते हैं। स्थिति के अनुसार एक बार में केवल एक ही विकल्प Execute होता है।

IF_ELIF_ELSE Statement का Syntax इस प्रकार है:

```
if condition:
    statement(s)
elif condition:
    statement(s)
elif condition:
    statement(s)
else:
    statement(s)
```

IF_ELIF_ELSE DEMONSTRATION

```
a=int(input('Enter a Number\t'))
if a>0:
print('It is a Positive Number')
elif a<0:
print('it is a Negative Number')
else:
print('It is a Zero')
```

OUTPUT

```
Enter a Number      34
It is a Positive Number
```

```
Enter a Number      -12
it is a Negative Number
```

```
Enter a Number       0
It is a Zero
```

TOPIC 6.2 REPETITION

किसी Condition के पूरा होने तक Statements के समूह को बार-बार दोहराने की प्रक्रिया को Repetition कहते हैं। Python Language में दो प्रकार के Repetition Statements हैं:

(A) FOR LOOP (B) WHILE LOOP

FOR LOOP

for Statement का उपयोग Values की एक Range या Sequence को Iterate करने के लिए किया जाता है। for Loop Range में प्रत्येक Item के लिए Execute होती है।

For Loop का Syntax: for <control-variable> in <sequence/items in range>: <statements inside body of the loop>

The Range() Function range() Python में एक Built-in Function है। range() Function का Syntax है: range([start], stop[, step]) इसका उपयोग दिए गए Start Value से Stop Value तक (Stop Value को छोड़कर) दिए गए Step Value के अंतर से Integers की एक List बनाने के लिए किया जाता है।

FOR LOOP DEMONSTRATION

```
start=1
end=int(input('Enter a Number\t'))
for even in range(start,end+1):
    if even%2==0:
        print(even,"\t")
```

OUTPUT

```
Enter a Number      20
2
4
6
8
10
12
14
16
18
```


WHILE LOOP

while Statement एक Block of Code को तब तक बार-बार Execute करता है जब तक Loop की Control Condition True रहती है। while Loop की Control Condition को Loop के अंदर किसी भी Statement के Execute होने से पहले जाँचा जाता है।

while Loop का Syntax: while test_condition: body of while

WHILE LOOP IMPLEMENTATION

```
start=1
num=int(input('Enter A Number'))
while start<=num:
    print(start,'\t')
    start=start+1
```

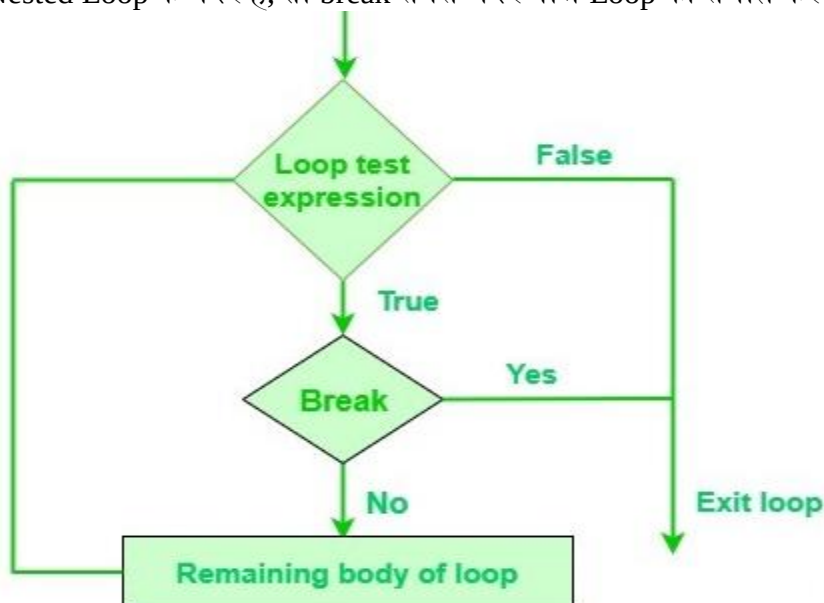
OUTPUT

```
Enter a Number      10
1
2
3
4
5
6
7
8
9
10
```

TOPIC 6.3 BREAK & CONTINUE STATEMENT

BREAK STATEMENT

Python में break Statement का उपयोग किसी बाहरी Condition के Trigger होने पर Loop से Control बाहर लाने के लिए किया जाता है। break Statement को Loop Body के अंदर (सामान्यतः if Condition के बाद) रखा जाता है। यह वर्तमान Loop को समाप्त करता है और Loop के तुरंत बाद के अगले Statement पर Execution फिर से शुरू होती है। यदि break Statement किसी Nested Loop के अंदर है, तो break सबसे अंदर वाले Loop को समाप्त करेगा।



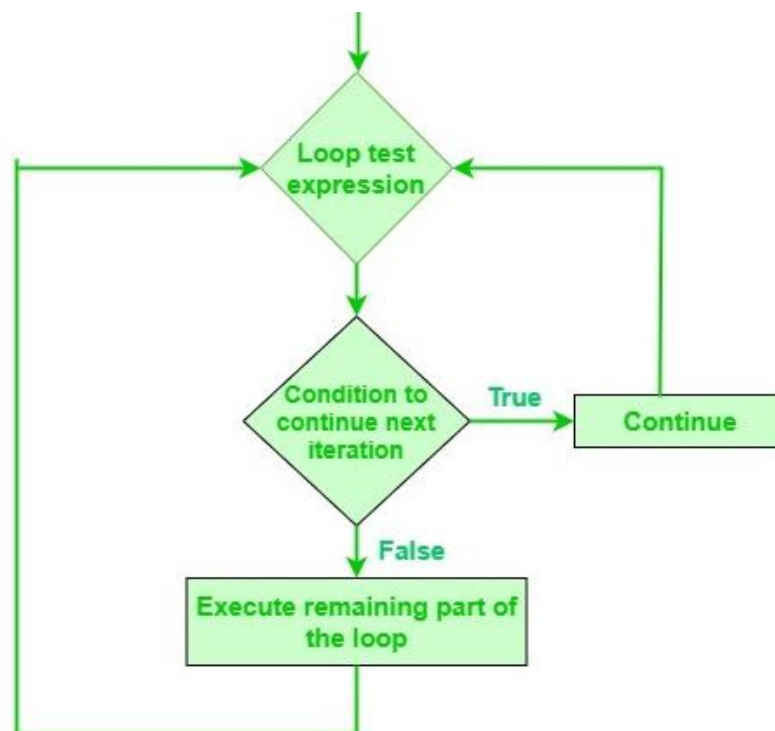
```
# Use of Break Statement
for i in range(10):
    print(i)
    if i==7:
        break
```

OUTPUT

```
0
1
2
3
4
5
6
7
```

CONTINUE STATEMENT

Python Continue Statement एक Loop Control Statement है जो Loop के Current Iteration के लिए Loop के अंदर शेष Code को Skip करके अगले Iteration को Execute करने के लिए बाध्य करता है। अर्थात जब Loop में continue Statement Execute होता है, तो उस Iteration के लिए continue Statement के बाद का Code Skip हो जाता है और Loop का अगला Iteration शुरू हो जाता है।



```
# Use of Continue Statement
for i in range(10):
    if i==7:
        continue
    print(i)
```

FUNCTIONS

TOPIC 7.1 WHAT IS FUNCTION?

Function एक छोटा Program होता है जिसका उपयोग Computer में किसी विशेष कार्य को करने के लिए किया जाता है। Function किसी भी Programming Language की सबसे शक्तिशाली विशेषता है। Python Language भी Functions में समृद्ध है। Function का उपयोग बड़े Programs को छोटे Tasks में विभाजित करने के लिए भी किया जाता है। Professional Development Environment में Functions का बहुत अधिक उपयोग होता है। Function Program Development Approach को तेज करता है और Computer Program की Speed बढ़ाता है। Function के तीन Components होते हैं:

- (1) Name of Function
- (2) Arguments & Parameters of Function
- (3) Body of Function

Arguments & Parameters of Function

Name of Function: किसी Function को Call और Execute करने के लिए दिया गया एक विशेष नाम।

Arguments of Function: Function Processing के लिए उपयोग किए जाने वाले Local Variables।

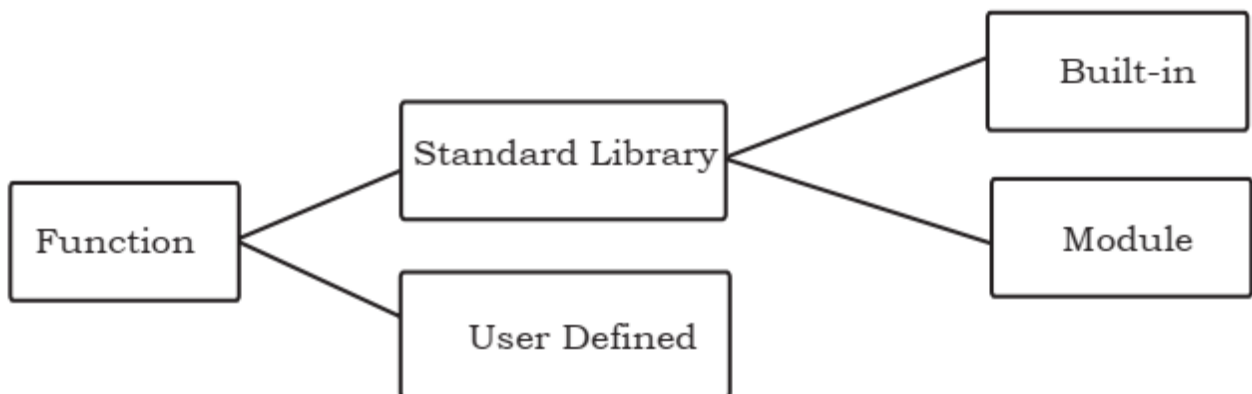
Parameters of Function: Function को दिए जाने वाले Input Variables।

Body of Function: Function का Processing Part।

TOPIC 7.2 ADVANTAGES OF FUNCTIONS

- (1) Function Program की Readability बढ़ाता है और उसे समझना आसान बनाता है।
- (2) Function Program की Code Length घटाता है और एक ही Program को बार-बार लिखने से बचाता है।
- (3) Function एक बड़े Program को छोटे Programs में विभाजित करता है।
- (4) Function Inheritance की आदत को बढ़ाता है।
- (5) कार्य को छोटी Teams में विभाजित किया जा सकता है और Parallel Processing की जा सकती है।

TOPIC 7.3 TYPES OF FUNCTIONS

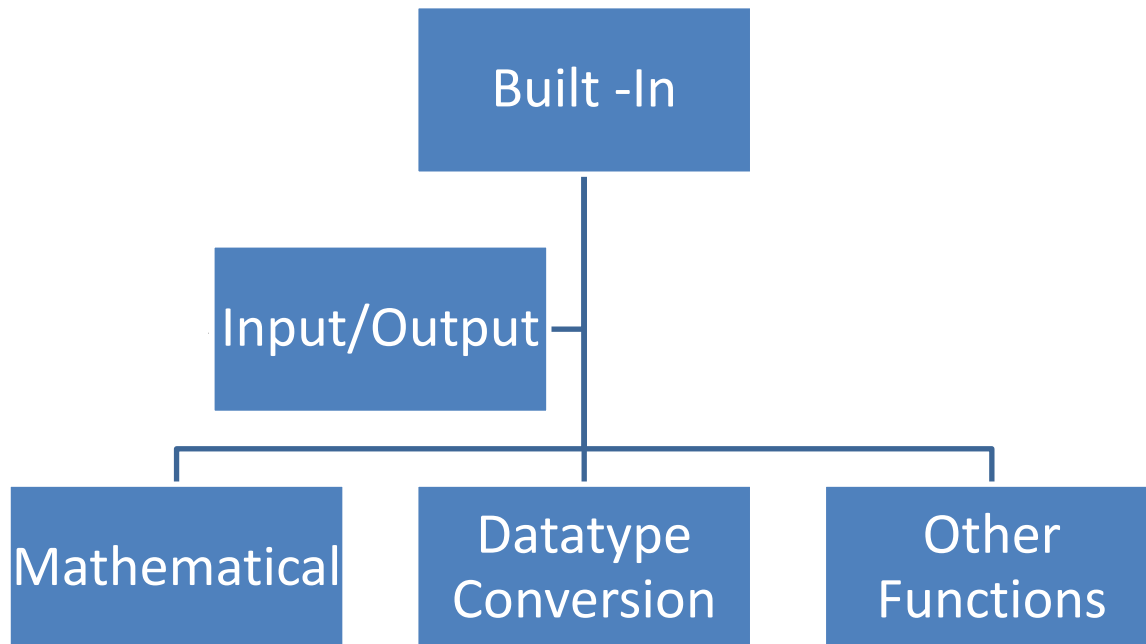


STANDARD LIBRARY

Python Language में विभिन्न Functions का एक संग्रह है जिसे किसी भी Program में आवश्यकतानुसार उपयोग किया जा सकता है। Python Standard Library में विभिन्न Built-in Functions और Modules शामिल हैं।

BUILT-IN FUNCTIONS

ये Ready-Made Functions होते हैं जिन्हें किसी भी Programming Language में सीधे उपयोग किया जा सकता है। विभिन्न Built-in Functions निम्नलिखित हैं:



INPUT/OUTPUT	MATHEMATICAL	DATATYPE CONVERSION	OTHER FUNCTIONS
INPUT ()	BOOL ()	ABS ()	LEN ()
PRINT ()	CHAR ()	MAX ()	RANGE ()
	DICT ()	MIN ()	TYPE ()
	INT ()	DIVMOD ()	
	FLOAT ()	POW ()	
	LIST ()	SUM ()	
	ORD ()		
	SET ()		
	STR ()		
	TUPLE ()		

FUNCTION	DESCRIPTION	EXAMPLE
abs(x)	x का Absolute Value (परम मान)	<pre>>>> abs(4) 4 >>> abs(-5.7) 5.7</pre>
divmod(x,y)	Quotient और Remainder का एक Tuple	<pre>>>> divmod(7,2) (3, 1)</pre>
max(sequence)	Largest Number in the Sequence	<pre>>>> max([1,2,3,4]) 4</pre>

FUNCTION	DESCRIPTION	EXAMPLE
<code>pow(x,y[,z])</code>	X और Y (X को Y की शक्ति तक उठाया गया) यदि Z दिया गया है, तो: $(x,y) \% z$	<pre>>>> pow(5,2) 25.0 >>> pow(5,2,4) 1</pre>
<code>sum(x[,num])</code>	Sequence के सभी Elements का योग (Left से Right)। यदि num Parameter दिया गया है, तो वह भी योग में जोड़ा जाएगा।	<pre>>>> sum([2,4,7,3]) 16 >>> sum([2,4,7,3],3) 19 >>> sum((52,8,4,2)) 66</pre>
<code>len(x)</code>	Count Number of Elements	<pre>>>> len("Patience") 8 >>> len([12,34,98]) 3</pre>

MODULE

Module Functions का एक समूह होता है। एक Program को विभिन्न स्तरों पर Modules कहे जाने वाले विभिन्न भागों में विभाजित किया जा सकता है।

एक Module, Function Definitions के संग्रह वाली Python File के रूप में बनाया जाता है।

किसी Module का उपयोग करने के लिए हमें उसे अपने Computer Program में Import करना होता है, फिर हम उस Module के सभी Functions का उपयोग कर सकते हैं। Module Import करने का Syntax इस प्रकार है:

import module1, module2 | module-name.function-name

Python Language में तीन प्रकार के Modules होते हैं:

- (1) MATH
- (2) RANDOM
- (3) STATISTICS

MATH MODULE

FUNCTION	DESCRIPTION	EXAMPLE
<code>math.ceil(x)</code>	x का Ceiling Value (ऊपरी पूर्णांक मान)	<pre>>>> math.ceil(-9.7) -9 >>> math.ceil(9.7) 10</pre>
<code>math.floor(x)</code>	x का Floor Value (निचला पूर्णांक मान)	<pre>>>> math.floor(-4.5) -5 >>> math.floor(4.5) 4</pre>

FUNCTION	DESCRIPTION	EXAMPLE
----------	-------------	---------

math.fabs(x)	x का Absolute Value (परम मान)	>>> math.fabs(-6.7) 6.7
math.factorial(x)	Factorial of x	>>> math.factorial(5) 120
math.fmod(x,y)	x का चिह्न रखते हुए $x \% y$	>>> math.fmod(4,4.9) 4.0
math.gcd(x,y)	x और y का GCD (Greatest Common Divisor / महत्तम समापवर्तक)	>>> math.gcd(10,2) 2
math.pow(x,y)	X और Y (X को Y की शक्ति तक उठाया गया)	>>> math.pow(3,2) 9.0
math.sqrt(x)	x का वर्गमूल (Square Root)	>>> math.sqrt(144) 12.0
math.sin(x)	Radians में x का Sine	>>> math.sin(6) -0.279

RANDOM MODULE

FUNCTION	DECIPTION	EXAMPLE
random.random()	0.0 से 1.0 की Range में Random Real Number (float)	>>> random.random() 0.65333522
random.randint(x,y)	x और y के बीच Random Integer, जहाँ $x \leq y$	>>> random.randint(3,7) 4
random.randrange(y)	0 और y के बीच Random Integer	>>> random.randrange(5) 4
random.randrange(x,y)	x और y के बीच Random Integer	>>> random.randrange(2,7) 2

STATISTICS MODULE

FUNCTION	DECIPTION	EXAMPLE
statistics.mean(x)	arithmetic mean	>>> statistics.mean([11,24,32,45,51]) 32.6
statistics.median(x)	x का Median (मध्य मान)	>>> statistics.median([11,24,32,45,51]) 32
statistics.mode(x)	Mode (सबसे अधिक बार आने वाला मान)	>>> statistics.mode([11,24,11,45,11]) 11

TOPIC 7.4 USER DEFINED FUNCTIONS

किसी User द्वारा अपने Programming Task को करने के लिए बनाया गया Function, User Defined Function कहलाता है। User Defined Function की शुरुआत def Keyword से होती है जिसके बाद Function का नाम होता है। User Defined Function Programmer के काम को बेहतर बनाता है और Code Reusability की सुविधा देता है।

SYNTAX

```
def functionname( argument1,argument2):  
    { set of intructions}  
    Return value  
functionname(parameter1,parameter2)
```

SAMPLE PROGRAM

```
# User Defined Function Area of Circle  
def areacircle(r):
```

```
    pie=3.14  
    a=pie*(r*r)  
    return a
```

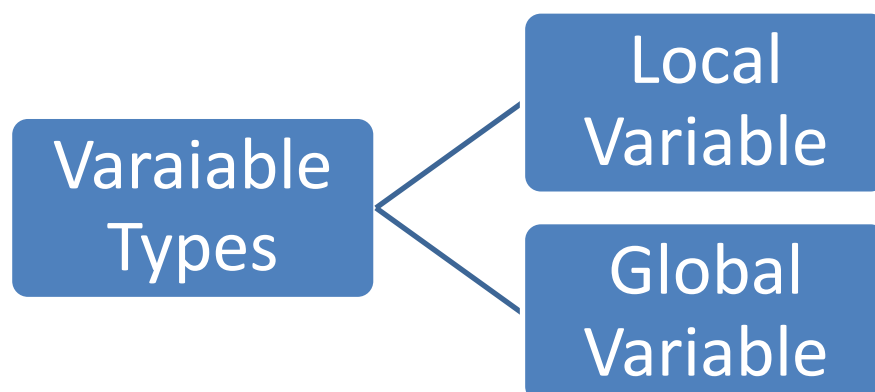
```
p=float(input('Enter Radius of Circle\t'))  
q=areacircle(p)  
print('Area of Circle',q)
```

OUTPUT

```
Enter Radius of Circle    3.5  
Area of Circle 38.465
```

TOPIC 7.5 SCOPE OF A VARIABLE

किसी भी Programming Language में Variable के काम करने का एक Area होता है, इसे Variable का Scope कहते हैं। इस अवधारणा के अनुसार दो प्रकार के Variables होते हैं: Local Variable और Global Variable।



LOCAL VARIABLE

किसी Function या Block के अंदर परिभाषित Variable को Local Variable कहते हैं। इसे केवल उसी Function या Block में Access किया जा सकता है जहाँ यह परिभाषित है। यह केवल तब तक Exist रहता है जब तक Function Execute होता है।

GLOBAL VARIABLE

किसी Function या Block के बाहर परिभाषित Variable को Global Variable कहते हैं। इसे आगे परिभाषित किसी भी Function में Access किया जा सकता है। Global Variable में किया गया कोई भी बदलाव Program के उन सभी Functions को प्रभावित करेगा जहाँ वह Variable Access किया जा सकता है।

SAMPLE PROGRAM

```
# Local & Global Variable
globe=50
def average(a,b,c):
    sum=a+b+c+globe
    avg=sum/4
    return avg
x=int(input('Enter First Number'))
y=int(input('Enter Second Number'))
z=int(input('Enter Third Number'))
p=average(x,y,z)
print('Average=',p)
```

OUTPUT

```
Enter First Number 10
Enter Second Number 20
Enter Third Number 30
Average= 27.5
```

LESSON 08 STRINGS

TOPIC 8.1 WHAT IS STRING?

String Characters का एक ऐसा संग्रह होता है जो एक Sequence में बंद होता है। String को Universal Code Characters भी कहते हैं। String को Single Quote या Double Quote में बंद किया जाता है।

Example: print('HELLO')

String के प्रत्येक Character को Index Number वाले Memory Cells में Store किया जाता है। शुरुआती Index Number 0 होता है और अंतिम Index Number n-1 होता है।

Positive Indices	0	1	2	3	4	5	6	7	8	9	10	11
String	H	e	l	l	o		W	o	r	l	d	!
Negative Indices	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

EXAMPLE

```
# String Implementation
ch=input('Enter Your Name\t')
print('Hello\t',ch)
```

OUTPUT

```
Enter Your Name    RAJ KAPOOR
Hello    RAJ KAPOOR
```

TOPIC 8.2 STRING OPERATIONS

SR.NO	OPERATION NAME	DESCRIPTION
1	Concatenation	Join Two or More String Together using + Operator
2	Membership	पता लगाना कि Sub String मुख्य String का भाग है या नहीं
3	Slicing	String के कुछ भाग को Access करना
4	Repetition	किसी दी गई String को * Operator का उपयोग करके कई बार Repeat करना

EXAMPLE

```
# String Operation
ch='The Internet is used for Communication'
str='It is Fast'
print('Concatenation')
print(ch,str)
print('Repetition')
print(str*3)
print('Slicing')
print(str[3:8])
print('Membership')
print('not used' in ch)
```

OUTPUT

```
Concatenation
The Internet is used for Communication It is Fast
Repetition
It is FastIt is FastIt is Fast
Slicing
is Fa
Membership
False
```

TOPIC 8.3 STRING FUNCTIONS

SR. NO	FUNCTION	DESCRIPTION	EXAMPLE
1	len ()	दी गई String में Characters की संख्या लौटाता है	>>> str1 = 'Hello World!' >> len(str1) 12
2	title ()	प्रत्येक शब्द के पहले अक्षर को Uppercase में बदलता है	>>> str1 = 'hello WORLD!' >> str1.title() 'Hello World!'

3	lower ()	दी गई String को Lowercase में बदलता है	>>> str1 = 'hello WORLD!' >>> str1.lower() 'hello world!'
4	upper ()	दी गई String को Uppercase में बदलता है	>>> str1 = 'hello WORLD!' >>> str1.upper() 'HELLO WORLD!'
5	count(str,start,end)	return number of times a sub string appears in a given string from start to end	>>> str1 = 'Hello World!' Hello Hello' >>> str1.count('Hello',12,25) 2
6	find(str,start,end)	दी गई String में Sub String की पहली Occurrence को Search करता है	>>> str1 = 'Hello World!' Hello Hello' >>> str1.find('Hello',10,20) 13
7	index(str,start,end)	यह find() की तरह काम करता है, लेकिन यदि Sub String नहीं मिलती तो Exception उत्पन्न करता है	>>> str1.index('Hee') ValueError: substring not found
8	endswith ()	True लौटाता है यदि दी गई String आपूर्त किए गए Substring से समाप्त होती है, अन्यथा False	>>> str1 = 'Hello World!' >>> str1.endswith('World!') True
9	startswith ()	True लौटाता है यदि दी गई String आपूर्त किए गए Substring से शुरू होती है, अन्यथा False	>>> str1 = 'Hello World!' >>> str1.startswith('He') True
10	isalnum ()	True लौटाता है यदि दी गई String के Characters Alphabets या Numeric हैं	>>> str1 = 'HelloWorld2' >>> str1.isalnum() True
11	islower()	True लौटाता है यदि दी गई String के Characters Lowercase में हैं	>>> str1 = 'Hello World!' >>> str1.islower() False

12	isupper()	True लौटाता है यदि दी गई String के Characters Uppercase में हैं	>>> str1 = 'HELLO WORLD!' >> str1.isupper() True
13	isspace()	True लौटाता है यदि String में Blank Spaces हैं	>>> str1 = 'Rajesh in Market' >> str1.isspace() True
14	istitle()	True लौटाता है यदि String Title Case में है	>>> str1 = 'Rajesh In Market' >> str1.istitle() True
15	replace(oldstr,newstr)	एक मौजूदा String को नई String से बदलता है	>>> str1 = 'Hello World!' >> str1.replace('World','Country') 'Hello Country!'
16	join ()	यह Function String के Characters को Separator (-) के साथ जोड़ता है	>>> str1 = ('HelloWorld!') >> str2 = '-' #separator >> str2.join(str1) 'H-e-l-l-o-W-o-r-l-d-!'
17	partition ()	Sub String के शुरुआती बिंदु से String को विभाजित करता है	>>> str1 = 'India is a Great Country' >> str1.partition('is') ('India ', 'is', ' a Great Country')
18	split ()	String के प्रत्येक शब्द को फैलाता (Spread) है	>>> str1 = 'India is a Great Country' >> str1.split() ['India','is','a','Great', 'Country']

LIST

TOPIC 9.1 WHAT IS LIST?

List एक ऐसी Data Structure है जो Mutable प्रकार की होती है। List का उपयोग किसी भी प्रकार जैसे Integer, Float, Alphabet आदि का Data Store करने के लिए किया जाता है। यह Mixed Values Store करने के लिए एक महत्वपूर्ण Data

Type है। List के प्रत्येक Element को Index Number वाले Memory Cells में Store किया जाता है। शुरुआती Index Number 0 होता है और अंतिम Index Number n-1 होता है।

Positive Index	0	1	2	3	4	5	6	7	8
List Elements	50	90	100	78	23	89	45	12	120
Negative Index	-9	-8	-7	-6	-5	-4	-3	-1	-1

EXAMPLE

```
# List Implementation
A1=[10,20,30,40,50]
A2=[10.1,10.2,10.3,10.4,10.5]
A3=[10,45.67,'HELLO']
print(A1)
print(A2)
print(A3)
```

OUTPUT

```
[10, 20, 30, 40, 50]
[10.1, 10.2, 10.3, 10.4, 10.5]
[10, 45.67, 'HELLO']
```

```
# List Mutation
A1=[10,20,30,40,50]
print('Before Mutation')
print(A1)
A1[2]=100
print('After Mutation')
print(A1)
```

OUTPUT

```
Mutation से पहले (Before Mutation)
[10, 20, 30, 40, 50]
After Mutation
[10, 20, 100, 40, 50]
```

TOPIC 9.2 LIST OPERATIONS

SR.NO	OPERATION NAME	DESCRIPTION
1	Concatenation	+ Operator का उपयोग करके दो या अधिक Lists को एक साथ जोड़ना
2	Membership	पता लगाना कि Element List का भाग है या नहीं
3	Slicing	List के कुछ भाग को Access करना

4	Repetition	* Operator का उपयोग करके दी गई List को कई बार Repeat करना
---	------------	---

```
# List Operation
A1=[10,20,30,40,50]
A2=[60,70,80,90,100]
print('Concatenation')
print(A1+A2)
print('Repeat')
print(A1*3)
print('Membership')
print(26 in A1)
print('Slicing')
print(A2[3:4])
```

OUTPUT

```
Concatenation
[10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
Repeat
[10, 20, 30, 40, 50, 10, 20, 30, 40, 50, 10, 20, 30, 40, 50]
Membership
False
Slicing
[90]
```

TOPIC 9.3 LIST BUILT IN FUNCTIONS

FUNCTION	DESCRIPTION	EXAMPLE
len()	List की Length (लंबाई) लौटाता है	>>> list1 = [10,20,30,40,50] >>> len(list1) 5
list()	यदि कोई Argument नहीं दिया गया तो एक Empty List बनाता है	>>> list1 = list() >>> list1

append()	List के अंत में Argument के रूप में दिए गए Single Element को जोड़ता है	>>> list1 = [10,20,30,40] >>> list1.append(50) >>> list1 [10, 20, 30, 40, 50]
extend()	दी गई List के Argument के प्रत्येक Element को List के अंत में जोड़ता है	>>> list1 = [10,20,30] >>> list2 = [40,50] >>> list1.extend(list2) >>> list1 [10, 20, 30, 40, 50]
insert()	List में किसी विशेष Index पर एक Element Insert करता है	>>> list1 = [10,20,30,40,50] >>> list1.insert(2,25) >>> list1 [10, 20, 25, 30, 40, 50]
count()	दिए गए Element के List में आने की संख्या लौटाता है	>>> list1 = [10,20,30,10,40,10] >>> list1.count(10) 3
index()	List में Element की पहली Occurrence का Index लौटाता है। यदि Element	>>> list1 = [10,20,30,20,40,10] >>> list1.index(20)

	मौजूद नहीं है तो ValueError उत्पन्न होता है।	1
remove()	List से दिए गए Element को हटाता है। यदि Element कई बार मौजूद है तो केवल पहली Occurrence हटाई जाती है।	>>> list1 = [10,20,30,40,50,30] >>> list1.remove(30) >>> list1 [10, 20, 40, 50, 30]
pop()	वह Element लौटाता है जिसका Index Parameter के रूप में दिया गया हो और उसे List से भी हटाता है।	>>> list1 = [10,20,30,40,50,60] >>> list1.pop(3) 40

reverse()	दी गई List में Elements के क्रम को उलट देता है	>>> list1 = [34,66,12,89,28,99] >>> list1.reverse() >>> list1 [99, 28, 89, 12, 66, 34]
sort()	दी गई List के Elements को In-place Sort करता है	>>> list1 = ['Tiger','Zebra','Lion', 'Cat', 'Elephant', 'Dog'] >>> list1.sort() >>> list1 ['Cat', 'Dog', 'Elephant', 'Lion', 'Tiger', 'Zebra']
sorted()	एक List को Parameter के रूप में लेता है और उन्हीं Elements की एक नई Sorted List बनाता है	>>> list1 = [23,45,11,67,85,56] >>> list2 = sorted(list1) >>> list1 [23, 45, 11, 67, 85, 56] >>> list2 [11, 23, 45, 56, 67, 85]
min() max() sum()	List का Minimum (सबसे छोटा) Element लौटाता है List का Maximum (सबसे बड़ा) Element लौटाता है List के सभी Elements का योग (Sum) लौटाता है	>>> list1 = [34,12,63,39,92,44] >>> min(list1) 12 >>> max(list1) 92 >>> sum(list1) 284

TOPIC 9.4 NESTED LIST

जब किसी मौजूदा List में एक List Element के रूप में प्रकट होती है तो उसे Nested List कहते हैं।

EXAMPLE

```
# Nested List
A1=[10,20,30,40,['A','B','C','D'],50]
print(A1)
```

OUTPUT

```
[10, 20, 30, 40, ['A', 'B', 'C', 'D'], 50]
```

TOPIC 9.5 COPYING LIST

```
# Copy List
A1=[10,20,30,40,50,60,70,80,90]
```

```
A2=list(A1)
print(A2)
```

OUTPUT

```
[10, 20, 30, 40, 50, 60, 70, 80, 90]
```

TOPIC 9.6 LIST AS ARGUMENT OF FUNTION

```
# List Funtion
def INCREASE(B1):
    for P in range(0,len(B1)):
        B1[P]=B1[P]+5
```

```
A1=[50,60,70,80,90,100]
print('Normal List')
print(A1)
INCREASE(A1)
print('Incremented List')
print(A1)
```

OUTPUT

```
Normal List
[50, 60, 70, 80, 90, 100]
Incremented List
[55, 65, 75, 85, 95, 105]
```

TUPLE & DICTIONARY

TOPIC 10.1 WHAT IS TUPLE?

Tuple विभिन्न Data Types जैसे Integer, Float या String वाले Elements का एक Sequence होता है। Tuple के Elements अल्पविराम द्वारा अलग किए जाते हैं और Round Brackets में बंद होते हैं। Tuples Immutable प्रकार के होते हैं, Tuple में Store किया गया Data बदला नहीं जा सकता।

EXAMPLE

```
# Tuple Implementation
T1=(10,20,30,40,50)
T2=(45.7,58.9,69.80)
T3=(100,210.05,'A','B')
print(T1)
print(T2)
print(T3)
print('Selected')
print(T2[2])
```

OUTPUT

```
(10, 20, 30, 40, 50)
(45.7, 58.9, 69.8)
(100, 210.05, 'A', 'B')
Selected
```

TOPIC 10.2 TUPLE OPERATIONS

SR.NO	OPERATION NAME	DESCRIPTION
1	Concatenation	+ Operator का उपयोग करके दो या अधिक Tuples को एक साथ जोड़ना
2	Membership	पता लगाना कि Element Tuple का भाग है या नहीं
3	Slicing	Tuple के कुछ भाग को Access करना
4	Repetition	* Operator का उपयोग करके दिए गए Tuple को कई बार Repeat करना

```
# Tuple Operation
A1=(10,20,30,40,50)
A2=(60,70,80,90,100)
print('Concatenation')
print(A1+A2)
print('Repeat')
print(A1*3)
print('Membership')
print(26 in A1)
print('Slicing')
print(A2[3:4])
```

OUTPUT

```
Concatenation
(10, 20, 30, 40, 50, 60, 70, 80, 90, 100)
Repeat
(10, 20, 30, 40, 50, 10, 20, 30, 40, 50, 10, 20, 30, 40, 50)
Membership
False
Slicing
(90,)
```

TOPIC 10.3 TUPLE BUILT IN METHODS

METHOD	DESCRIPTION	EXAMPLE
len()	Tuple की Length या Elements की संख्या लौटाता है	>>> tuple1 = (10,20,30,40,50) >>> len(tuple1) 5
tuple()	यदि कोई Argument नहीं दिया गया तो एक Empty Tuple बनाता है	>>> tuple1 = tuple() >>> tuple1 ()
count()	Tuple में दिए गए Element के आने की संख्या लौटाता है	>>> tuple1 = (10,20,30,10,40,10,50) >>> tuple1.count(10) 3

index()	Tuple में Element की पहली Occurrence का Index लौटाता है	>>> tuple1 = (10,20,30,40,50) >>> tuple1.index(30) 2
sorted()	Tuple के Elements लेता है और एक नई Sorted List लौटाता है	>>> tuple1 = ("Rama","Heena","Raj", "Mohsin","Aditya") >>> sorted(tuple1) ['Aditya', 'Heena', 'Mohsin', 'Raj', 'Rama']
min() max() sum()	Tuple का Minimum (सबसे छोटा) Element लौटाता है Tuple का Maximum (सबसे बड़ा) Element लौटाता है Tuple के सभी Elements का योग लौटाता है	>>> tuple1 = (19,12,56,18,9,87,34) >>> min(tuple1) 9 >>> max(tuple1) 87 >>> sum(tuple1) 235

TOPIC 10.4 NESTED TUPLE

जब किसी मौजूदा List में एक Tuple Element के रूप में प्रकट होता है तो उसे Nested Tuple कहते हैं।

Nested Tuple

```
A1=(10,20,30,40,('A','B','C','D'),50)
```

```
print(A1)
```

TOPIC 10.5 INTRODUCTION TO DICTIONARIES

Dictionary Data Type, Mapping के अंतर्गत आता है। यह Keys के Set और Values के Set के बीच एक Mapping है। Key-Value Pair को Item कहते हैं। एक Key को Colon (:) द्वारा अपने Value से अलग किया जाता है और बाकी Items को अल्पविराम द्वारा। Dictionary Mutable प्रकार की होती है। हम नए Elements जोड़ सकते हैं और मौजूदा Elements को Modify कर सकते हैं।

Dictionary Implementation

```
D1={'Wheat':13,'Rice':18,'Mango':26,'Orange':18,'Banana':10}
```

```
print(D1)
```

```
print('Selected Item')
```

```
print(D1['Mango'])
```

```
print('Add New Item')
```

```
D1['Dragon Fruit']=56
```

```
print(D1)
```

```
print('Modify Element')
```

```
D1['Orange']=23
```

```
print(D1)
```

OUTPUT

```
{'Wheat': 13, 'Rice': 18, 'Mango': 26, 'Orange': 18, 'Banana': 10}
```

```
Selected Item
```

```
26
```

```
Add New Item
```

```
{'Wheat': 13, 'Rice': 18, 'Mango': 26, 'Orange': 18, 'Banana': 10, 'Dragon Fruit': 56}
```

```
Modify Element
```

```
{'Wheat': 13, 'Rice': 18, 'Mango': 26, 'Orange': 23, 'Banana': 10, 'Dragon Fruit': 56}
```

TOPIC 10.6 DICTIONARY OPERATION: MEMBERSHIP

Membership Operator in यह Check करता है कि Key Dictionary में मौजूद है या नहीं। यदि Key मिलती है तो True, अन्यथा False लौटाता है।

```
>>> dict1 = {'Mohan':95,'Ram':89,'Suhel':92, 'Sangeeta':85}
```

```
>>> 'Suhel' in dict1
```

True

not in Operator True लौटाता है यदि Key Dictionary में मौजूद नहीं है, अन्यथा False।

```
>>> dict1 = {'Mohan':95,'Ram':89,'Suhel':92,'Sangeeta':85}
```

```
>>> 'Suhel' not in dict1
```

False

TOPIC 10.7 DICTIONARY BUILT IN METHODS

METHOD	DESCRIPTION	EXAMPLE
len()	Dictionary की Length या key:value Pairs की संख्या लौटाता है	<pre>>>> dict1 = {'Mohan':95,'Ram':89, 'Suhel':92, 'Sangeeta':85} >>> len(dict1) 4</pre>
dict()	Key-Value Pairs के Sequence से Dictionary बनाता है	<pre>pair1 = [('Mohan',95),('Ram',89), ('Suhel',92),('Sangeeta',85)] >>> dict1 = dict(pair1) >>> dict1 {'Mohan': 95, 'Ram': 89, 'Suhel': 92, 'Sangeeta': 85}</pre>
keys()	Dictionary में Keys की List लौटाता है	<pre>>>> dict1 = {'Mohan':95, 'Ram':89, 'Suhel':92, 'Sangeeta':85} >>> dict1.keys() dict_keys(['Mohan', 'Ram', 'Suhel', 'Sangeeta'])</pre>
values()	Dictionary में Values की List लौटाता है	<pre>>>> dict1 = {'Mohan':95, 'Ram':89, 'Suhel':92, 'Sangeeta':85} >>> dict1.values() dict_values([95, 89, 92, 85])</pre>
items()	Tuples (key-value) pairs की List लौटाता है	<pre>>>> dict1 = {'Mohan':95, 'Ram':89, 'Suhel':92, 'Sangeeta':85} >>> dict1.items() dict_items([('Mohan', 95), ('Ram', 89), ('Suhel', 92), ('Sangeeta', 85)])</pre>
get()	Argument के रूप में दी गई Key के अनुरूप Value लौटाता है	<pre>>>> dict1 = {'Mohan':95, 'Ram':89, 'Suhel':92, 'Sangeeta':85} >>> dict1.get('Sangeeta') 85</pre>

update()	Argument के रूप में दी गई Dictionary के Key-Value Pairs जोड़ता है	<pre>>>> dict1 = {'Mohan':95, 'Ram':89, 'Suhel':92, 'Sangeeta':85} >>> dict2 = {'Sohan':79,'Geeta':89} >>> dict1.update(dict2) >>> dict1 {'Mohan': 95, 'Ram': 89, 'Suhel': 92, 'Sangeeta': 85, 'Sohan': 79, 'Geeta': 89}</pre>
clear()	Dictionary के सभी Items को Delete या Clear करता है	<pre>>>> dict1 = {'Mohan':95,'Ram':89, 'Suhel':92, 'Sangeeta':85} >>> dict1.clear()</pre>
del()	दी गई Key वाले Item को Delete करता है	<pre>>>> dict1 = {'Mohan':95,'Ram':89,'Suhel':92, 'Sangeeta':85} >>> del dict1['Ram'] >>> dict1 {'Mohan':95,'Suhel':92, 'Sangeeta':85}</pre>

EXCEPTION HANDLING

TOPIC 1.1 INTRODUCTION

कुछ स्थितियों में Computer Program असामान्य रूप से समाप्त हो जाता है। यह समाप्ति Program में आई अप्रत्याशित Error के कारण होती है। Exception Handling, Errors से निपटने का एक तंत्र (Mechanism) है। Exception Handling Errors को संभालती है और Program की असामान्य समाप्ति को रोकती है। Computer Terminology में तीन प्रकार की Errors होती हैं। Python Language में Exceptions वे Errors होती हैं जो स्वतः Trigger होती हैं।

SYNTAX ERROR

इस प्रकार की Error तब होती है जब Programmer Programming Language के नियमों और विनियमों का पालन नहीं करता। इसे Parsing Error भी कहते हैं। Syntax Error में Interpreter Program को तब तक Execute नहीं करता जब तक Error हल न हो जाए।

LOGICAL ERROR

Logic Error Program के Source Code में एक ऐसी Error होती है जो अप्रत्याशित और गलत व्यवहार को जन्म देती है। Logic Error एक ऐसी Error है जो Program को गलत Output दे सकती है। यह Program के चलते समय Crash का कारण भी बन सकती है।

RUN TIME ERROR

Runtime Error एक ऐसी Error होती है जो Program के सफलतापूर्वक Compile होने के बाद, Program चलते समय आती है।

Runtime Errors को सामान्यतः 'bugs' कहा जाता है और ये अक्सर Software Release होने से पहले Debugging Process के दौरान पाई जाती हैं।

जब किसी Public Program में Runtime Errors आती हैं, तो Developers अक्सर Patches जारी करते हैं — ये छोटे Updates होते हैं जो Errors को ठीक करने के लिए बनाए जाते हैं।

TOPIC 1.2 EXCEPTION

Exception एक Python Object होती है जो एक Error को दर्शाती है। जैसे ही Program चलते समय कोई Error आती है, Exception Raised होती है। इस प्रकार की Exception को Programmer द्वारा Handle किया जाना चाहिए। यदि Error को Handle किया गया तो Program असामान्य रूप से Terminate नहीं होगा। Exception Handling Program को Exception से सुरक्षित रखती है।

TOPIC 1.3 BUILT IN EXCEPTION

Python Language में Pre-defined Exceptions की एक समृद्ध Library है जिसका उपयोग विभिन्न प्रकार की Exceptions को Manage करने के लिए किया जा सकता है। Python Language में 12 प्रकार के Built-in Exceptions उपलब्ध हैं। Exception Handlers User को आई हुई Error के बारे में उचित Message प्रदर्शित करते हैं।

S. No	Name of the Built-in Exception	Explanation
1.	SyntaxError	It is raised when there is an error in the syntax of the Python code.
2.	ValueError	It is raised when a built-in method or operation receives an argument that has the right data type but mismatched or inappropriate values.
3.	IOError	It is raised when the file specified in a program statement cannot be opened.
4	KeyboardInterrupt	It is raised when the user accidentally hits the Delete or Esc key while executing a program due to which the normal flow of the program is interrupted.
5	ImportError	It is raised when the requested module definition is not found.
6	EOFError	It is raised when the end of file condition is reached without reading any data by input().
7	ZeroDivisionError	It is raised when the denominator in a division operation is zero.
8	IndexError	It is raised when the index or subscript in a sequence is out of range.
9	NameError	It is raised when a local or global variable name is not defined.
10	IndentationError	It is raised due to incorrect indentation in the program code.
11	TypeError	It is raised when an operator is supplied with a value of incorrect data type.
12	OverflowError	It is raised when the result of a calculation exceeds the maximum limit for numeric data type.

TOPIC 1.4 THE RAISE STATEMENT

The raise statement is used to throw an exception. It works for both user defined and built in exception. If an exception is generated then raise statement show the error message as string.

Syntax

```
raise exception-name[(optional argument)]
```

Example

```
# Raise Statement
num=[10,20,30,40,50]
max=10
if max > len(num):
    raise IndexError
    print('Program Not Run')
else:
```



```
print(max)
```

OUTPUT

Traceback (most recent call last):

File "C:/Users/harsh/Documents/SAMPLE.py", line 5, in <module>

raise IndexError

IndexError

TOPIC 1.5 THE ASSERT STATEMENT

The Assert statement is used to test to an expression whether it can generate an exception or not. If the result of the test is false then the exception is raised.

Assert Statement

```
def check(num):
```

```
    assert num>=0,print('It is a Negative Number')
```

```
    print(num*num)
```

```
a=int(input('Enter A Number\t'))
```

```
check(a)
```

OUTPUT

```
Enter A Number      -12
```

```
It is a Negative Number
```

Traceback (most recent call last):

File "C:/Users/harsh/Documents/SAMPLE.py", line 7, in <module>

check(a)

File "C:/Users/harsh/Documents/SAMPLE.py", line 3, in check

assert num>=0,print('It is a Negative Number')

AssertionError: None

TOPIC 1.6 NEED FOR EXCEPTION HANDLING

Exception Handling की अवधारणा Ruby, JAVA, C++ के साथ-साथ Python Language में भी उपलब्ध है।

Exception Handling का उपयोग Runtime Errors को Handle करने के लिए किया जाता है ताकि Program बिना किसी Crash के सुचारू रूप से चल सके। Exception Handling की आवश्यकता इसलिए है क्योंकि:

Python Exceptions को अलग-अलग प्रकारों में वर्गीकृत करती है ताकि विशिष्ट Exception Handlers काम कर सकें।

Exception Handlers Program के मुख्य Logic को Error Detection और Correction Code से अलग करते हैं। Code का वह भाग जहाँ Error या Exception की संभावना होती है, उसे एक Block में रखा जाता है।

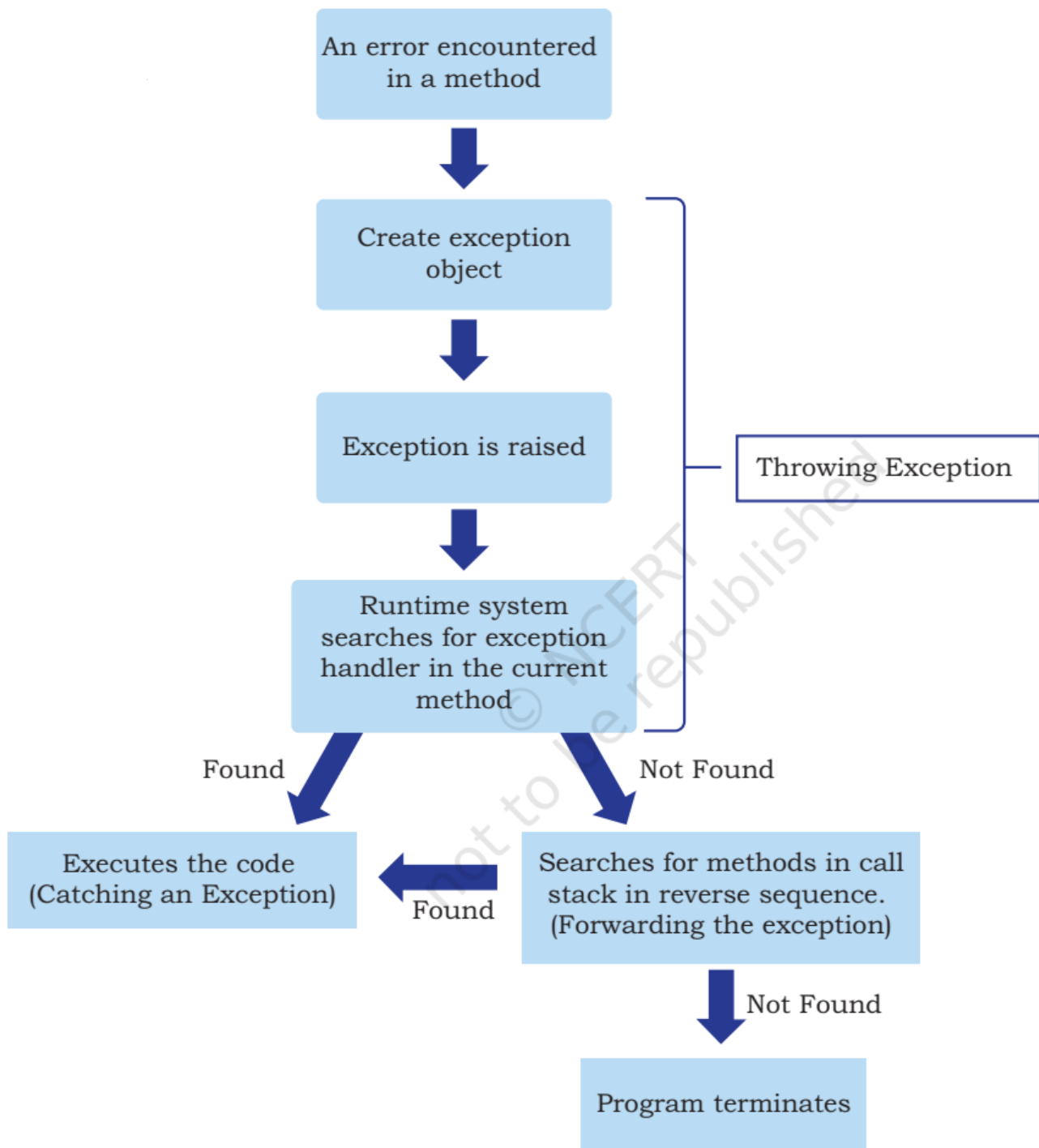
Compiler या Interpreter उस सटीक स्थान का रिकॉर्ड रखता है जहाँ Error हुई है।

Exception Handling, User-Defined और Built-In दोनों प्रकार की Exceptions के लिए की जा सकती है।

TOPIC 1.7 PROCESS OF EXCEPTION HANDLING

Program चल रहा होता है और अचानक एक Error आ जाती है। Error आने पर Language Translator Exception का Object बनाता है। Error Object को Runtime Handler को सौंप दिया जाता है। Runtime Exception Handler सही

Code की तलाश करता है जो Exception को Manage करने में सक्षम हो। जब Error Manager मिल जाता है तो Exception Object उसे सौंप दिया जाता है, Exception Handle हो जाती है और Program Crash से बच जाता है। दूसरी ओर यदि Error Manager नहीं मिलता तो Program Terminate हो जाता है।



TOPIC 1.8 TRY_EXCEPT_ELSE BLOCK

किसी विशेष प्रकार की Error को Handle करने के लिए बनाए गए Program Code को Exception Handler कहते हैं। Program का वह भाग जिसमें Error आ सकती है, try Block में लिखा जाता है और Error Handling Block को except Block में लिखा जाता है। Syntax इस प्रकार है:

```
try:
[ program statements where exceptions might occur]
except [exception-name]:
[ code for exception handling if the exception-name error is
encountered]
```

Exception Handling

```
try:
    n=int(input('Enter Numerator\t'))
    d=int(input('Enter Denominator\t'))
    r=n/d
except ZeroDivisionError:
    print('The Division with Zero is Not Allowed')
except ValueError:
    print('The Number must be a Whole Number')
else:
    print('Result=',r)
```

OUTPUT

```
Enter Numerator    12
Enter Denominator  2
Result= 6.0

Enter Numerator    12
Enter Denominator  0
The Division with Zero is Not Allowed

Enter Numerator    3.4
The Number must be a Whole Number
```

TOPIC 1.9 FINALLY CLAUSE

Python में try Statement में एक Optional finally Clause भी हो सकता है। finally Block के अंदर के Statements हमेशा Execute होते हैं, चाहे try Block में कोई Exception आई हो या नहीं।

Exception Handling

```
try:
    n=int(input('Enter Numerator\t'))
    d=int(input('Enter Denominator\t'))
    r=n/d
except ZeroDivisionError:
    print('The Division with Zero is Not Allowed')
except ValueError:
    print('The Number must be a Whole Number')
else:
    print('Result=',r)
finally:
    print('Program Completed')
```

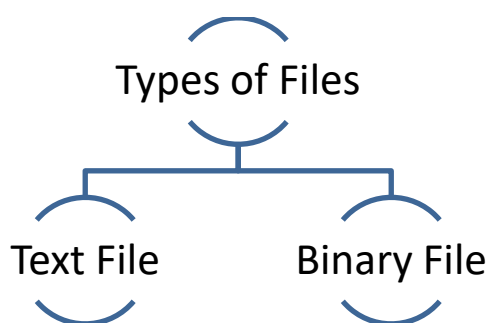
OUTPUT

```
Enter Numerator    12
Enter Denominator   2
Result= 6.0
Program Completed
```

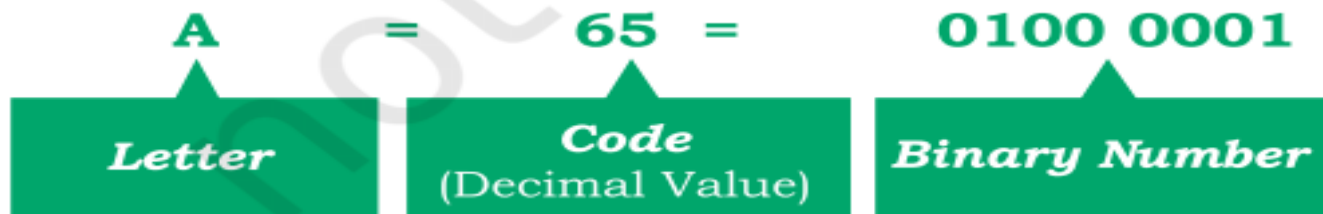
FILE HANDLING IN PYTHON

TOPIC 2.1 WHAT IS FILE?

File एक Container होती है जिसका उपयोग Data और Information Store करने के लिए किया जाता है। प्रत्येक File का एक Unique Name होता है। File Secondary Storage Media पर एक Named Location होती है जहाँ Data स्थायी रूप से Store रहता है।



TEXT FILE: - Text File को Characters का एक Sequence माना जा सकता है जिसमें Alphabets, Numbers और अन्य Special Symbols होते हैं। .txt, .py, .csv आदि Extensions वाली Files Text Files के उदाहरण हैं। प्रत्येक Text File में Data ASCII Code या UNICODE में Store होता है।



Text File की प्रत्येक पंक्ति (Line) End of Line (EOL) से समाप्त होती है। Python Language में Default EOL Character '\n' है।

BINARY FILE: - Binary Code (0, 1) वाले Bytes के Sequence को Binary File कहते हैं। यह File मानव पठनीय (Human Readable) रूप में नहीं होती। Binary File का उपयोग Private Information Store करने के लिए किया जाता है। Graphics, Audio, Video और Executable Files Binary Files होती हैं।

TOPIC 2.2 OPENING & CLOSING FILE

open() Function का उपयोग Computer Memory में File खोलने के लिए किया जाता है। यह दो Arguments — Filename और File Opening Mode — लेता है। Close() Function का उपयोग File बंद करने और उसे Memory से हटाने के लिए किया जाता है। किसी भी File पर काम करने से पहले उसे Open करना आवश्यक है। काम पूरा होने के बाद File को बंद करना आवश्यक है।

OPEN ()

```
file_object= open(file_name, access_mode)
```

```
x=open('myfile.txt','r')
```

```
CLOSE ( )
```

```
file_object.close( )
```

```
x.close( )
```

TOPIC 2.3 FILE OPENING MODE

File Mode	Description	File Offset position
<r>	Opens the file in read-only mode.	Beginning of the file
<rb>	Opens the file in binary and read-only mode.	Beginning of the file
<r+> or <+r>	Opens the file in both read and write mode.	Beginning of the file
<w>	Opens the file in write mode. If the file already exists, all the contents will be overwritten. If the file doesn't exist, then a new file will be created.	Beginning of the file
<wb+> or <+wb>	Opens the file in read,write and binary mode. If the file already exists, the contents will be overwritten. If the file doesn't exist, then a new file will be created.	Beginning of the file
<a>	Opens the file in append mode. If the file doesn't exist, then a new file will be created.	End of the file
<a+> or <+a>	Opens the file in append and read mode. If the file doesn't exist, then it will create a new file.	End of the file

TOPIC 2.4 FILE READ & WRITE METHODS

WRITE() METHOD

write Method का उपयोग File में एक Single Line लिखने के लिए किया जाता है। यह String को Argument के रूप में लेता है और उसे File में Pass करता है। File में Line का अंत दिखाने के लिए हमें New Line Character (\n) जोड़ना होता है। यदि कोई Numerical Data File में Pass करना है तो उसे पहले String में बदलना होगा। write() Method वास्तव में Buffer पर Data लिखता है और जब close Method Execute होता है तब Content File में लिखा जाता है।

SYNTAX

```
file_object=open(filename,'w')      file_object.write(data)      file_object.close( )
```

WRITELINES() METHOD

writelines Method का उपयोग File में Multiple Lines लिखने के लिए किया जाता है। यह List या Tuple से String को Argument के रूप में लेता है और File में Pass करता है।

SYNTAX

```
file_object=open(filename,'w')      file_object.writelines(data)      file_object.close( )
```

READ() METHOD

read Method का उपयोग File से निर्दिष्ट Bytes या सम्पूर्ण Data पढ़ने के लिए किया जाता है।

SYNTAX

```
file_object=open(filename,'r')          file_object.read( )          file_object.close( )
```

READLINE(n) METHOD

readline Method का उपयोग File से एक Complete Line पढ़ने के लिए किया जाता है, जहाँ प्रत्येक Line एक New Line Character से समाप्त होती है। यह New Line Character तक Maximum तय Bytes भी File से पढ़ सकता है।

SYNTAX

```
file_object=open(filename,'r')          file_object.readline(10 )          file_object.close( )
```

READLINES() METHOD

readlines Method का उपयोग File की सभी Lines को New Line के साथ पढ़ने के लिए किया जाता है। File की पूरी सामग्री Strings की List के रूप में Return होती है।

SYNTAX

```
file_object=open(filename,'r')          file_object.readlines( )          file_object.close( )
```

```
# File Read/Write Operation
data=['Raj is going to market\n','He is purchasing his favourite game\n']
x=open('myfile.txt','w')
x.writelines(data)
x.close( )
x=open('myfile.txt','r')
print(x.read( ))
x.close( )
```

OUTPUT

```
Raj is going to market
He is purchasing his favourite game
```

TOPIC 2.5 SETTING OFFSET

TELL() METHOD

यह Function एक Integer Value Return करता है जो File में File Pointer की वर्तमान स्थिति को दर्शाता है।
Syntax: file_object.tell()

SEEK() METHOD

इस Method का उपयोग किसी File में एक विशेष Position पर File Object को Locate करने के लिए किया जाता है।
seek() का Syntax है: file_object.seek(offset [, reference_point])
seek Method के निम्नलिखित Reference Values हैं:

0 - File की शुरुआत (Beginning) 1 - File की वर्तमान Position 2 - File का अंत (End)

Tell & Seek Method

```
data=input('Enter a Sentence\t')
x=open('myfile.txt','w')
x.write(data)
x.close( )
```

```
x=open('myfile.txt','r')
print('File Pointer',x.tell())
print(x.read( ))
x.seek(7,0)
print(x.read( ))
x.close( )
```

OUTPUT

```
Enter a Sentence    TODAY IS A RAINY DAY
File Pointer 0
TODAY IS A RAINY DAY
S A RAINY DAY
```

TOPIC 2.6 THE PICKLE MODULE

Python एक Module प्रदान करता है जिसे Pickle कहते हैं। Pickle Module का उपयोग किसी भी Python Object Structure को Serialize और De-Serialize करने के लिए किया जाता है। Serialization वह प्रक्रिया है जिसमें Memory (RAM) में Data या Object को Byte Streams में Transform किया जाता है। De-Serialization में Byte Stream को वापस Python Object में Convert किया जाता है।

DUMP() METHOD

इस Method का उपयोग Binary File में Data लिखने के लिए किया जाता है। जिस File में Data Dump करना है उसे Binary Write Mode (wb) में Open करना होता है। dump() का Syntax: dump(data_object, file_object)

LOAD() METHOD

इस Method का उपयोग Binary File से Data Load करने के लिए किया जाता है। जिस File को Load करना है उसे Binary Read (rb) Mode में Open किया जाता है। load() का Syntax: Store_object = load(file_object)

```
# Pickle Module
import pickle
itemno=int(input('Enter Item Number\t'))
itemname=input('Enter Item Name\t')
qty=int(input('Enter Quantity\t'))
price=float(input('Enter Price Per Item\t'))
amt=qty*price
data=[itemno,itemname,qty,price,amt]
x=open('myfile.dat','wb')
pickle.dump(data,x)
x.close( )
x=open('myfile.dat','rb')
r=pickle.load(x)
print(r)
x.close( )
```

OUTPUT

```
Enter Item Number    1001
Enter Item Name      RED PENCIL
Enter Quantity 48
Enter Price Per Item  2
[1001, 'RED PENCIL', 48, 2.0, 96.0]
```